Building VPNs on OpenBSD

Author: Daniele Mazzocchio
Last update: Jun 2, 2009
Latest version: http://www.kernel-panic.it/openbsd/vpn/

Table of Contents

1. Introduction....2
2. Ipsec overview....3
  2.1 IPsec protocols....3
  2.2 SA, SPI, SPD and other acronyms....4
  2.3 The life of an IPsec packet....5
3. Ipsec on OpenBSD....7
  3.1 Preliminary steps....7
  3.2 Setting up the PKI....8
  3.3 Configuration....9
  3.4 Packet filtering....12
  3.5 Redundant VPNs with sasyncd(8)....13
4. OpenVPN....15
  4.1 Installation and configuration....15
    4.1.1 Setting up the PKI....15
    4.1.2 Server configuration....18
    4.1.3 Client configuration....19
  4.2 Starting the VPN....21
5. OpenSSH....22
  5.1 Configuration....22
  5.2 Starting the VPN....23
  5.3 Finishing touches....23
6. Appendix....25
  6.1 References....25
  6.2 Bibliography....25

1. Introduction

A *VPN* is a network made up of multiple private networks situated at different locations, linked together using secure tunnels over a public (insecure) network, typically the Internet. Traffic inside VPN tunnels is usually encrypted and authenticated to provide security equivalent to that provided by leased lines, but at a fraction the cost. A *tunnel* is created by encapsulating a network protocol (e.g. IP) within another network protocol, operating at the same layer of the OSI model (e.g. IP, ICMP) or at a higher layer (e.g. ESP, TLS).

VPNs are becoming increasingly popular, as they allow companies to join the LANs of their branches or subsidiaries into a single private network (*site-to-site VPNs*) or to provide mobile employees, such as sales people, access to their corporate network from outside the premises (*remote-access VPNs*), thus making accessing and sharing internal information much easier.

Though most often associated with IPsec, VPNs are a rather broad concept and can be implemented using a number of different tunneling protocols (L2TP, MPLS, PPTP, TLS, among others). In particular, in this document, we will take a look at the three most popular VPN implementations supported by OpenBSD:

IPsec
: a suite of standard protocols, defined in various RFCs (see Appendix), that operate at the network layer of the OSI model; OpenBSD natively supports IPsec protocols and provides specific tools and daemons to manage IPsec VPNs;

OpenVPN
: an SSL-based VPN solution, operating at the application layer and probably the strongest contender for IPsec, thanks to its robustness, ease of use and portability;

OpenSSH
: since release 4.3, OpenSSH supports the tunneling of "*arbitrary network packets over a connection between an OpenSSH client and server, as a true VPN*" (see [OBSD39]).

Besides the inherent differences in cryptographic algorithms and authentication mechanims, these three VPN implementations differ under several aspects; each one has its own advantages and drawbacks and the choice among them must consider not only the ease of installation and administration, but also factors like bandwidth, reliability and scalability. The following are the most relevant differences:

- IPsec runs in kernel space, tightly integrated with the host TCP/IP stack, while OpenVPN and OpenSSH are user-space daemons. The in-kernel architecture has the advantage of being faster and more efficient, but may increase the impact of possible vulnerabilities and programming errors on the whole system;
- OpenVPN and OpenSSH have a slightly higher overhead due to the encapsulation of the payload within higher layers of the OSI model;
- IPsec works at the network layer of the OSI model, while both OpenVPN and OpenSSH can operate in either bridging mode (layer 2) or routing mode (layer 3) (please refer to [OVPN-FAQ] for a brief overview of bridging vs. routing); to tunnel ethernet traffic over IPsec, you need the additional layer of tunneling provided by the `gif(4)` interface;
- IPsec interoperability comes from its being a standard, but different vendors' implementations may not be entirely compatible; the interoperability of OpenVPN and OpenSSH, instead, is ensured by their high portability across the most popular OSes.

Despite the many differences, OpenVPN has some common ground with IPsec, since, as stated in [OVPN-SEC], OpenVPN's security model is heavily based on the IPSec ESP protocol for secure tunnel transport over UDP.

This document assumes that you are familiar with OpenBSD, since it won't cover topics like base system configuration, packages/ports installation or Packet Filter syntax.

2. Ipsec overview

IPsec configuration on OpenBSD is a pretty easy and straightforward process, especially compared to most other implementations; nevertheless, IPsec is a rather complicated beast and a good working knowledge of its protocols and internals is essential to configure it and get it to work properly. Therefore, before beginning the configuration, let's take a brief tour of the IPsec protocols and features.

IPsec (IP security) is a suite of standard protocols "*designed to provide interoperable, high quality, cryptographically-based security*" [RFC4301] for protecting communications over IPv4 and IPv6 networks. The main security services offered by IPsec are:

Confidentiality
: traffic is encrypted to ensure that only the legitimate receiver is able to access the data transmitted.

Connectionless integrity
: ensures that no modifications were made to the data while in transit across the network.

Data origin authentication
: the receiver is able to verify that data actually originates from the claimed source.

Detection and rejection of replays
: duplicate IP datagrams are detected and processed only once.

These security services are provided at the IP layer (layer 3 of the OSI model), thus protecting all protocols that may be carried over IP, including IP itself.

2.1 IPsec protocols

Most of IPsec security services are provided using two traffic security protocols:

AH (*Authentication Header*)
: defined in [RFC4302], AH is used to provide connectionless integrity, data origin authentication and optional (at the discretion of the receiver) anti-replay protection for IP datagrams.

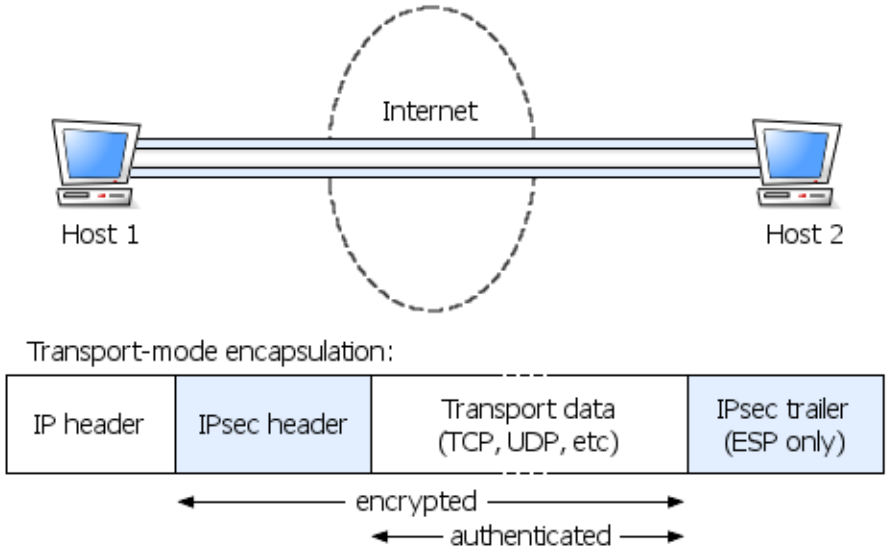
ESP (*Encapsulating Security Payload*)
: defined in [RFC4303], ESP offers the same set of services as AH (data origin authentication, connectionless integrity and anti-replay), plus confidentiality.

ESP is by far the most popular of the two protocols, since it provides confidentiality by encrypting network traffic, thus protecting transmitted data from passive attacks. On the other hand, AH provides stronger authentication than ESP as it protects part of the outer IP header as well as the next level protocol data, while ESP only protects the inner (encapsulated) IP header; however, this feature, in addition to not being of great use in most cases, also violates the modularization of the protocol stack (see [SCHNEIER], where the AH protocol is proposed for complete elimination).

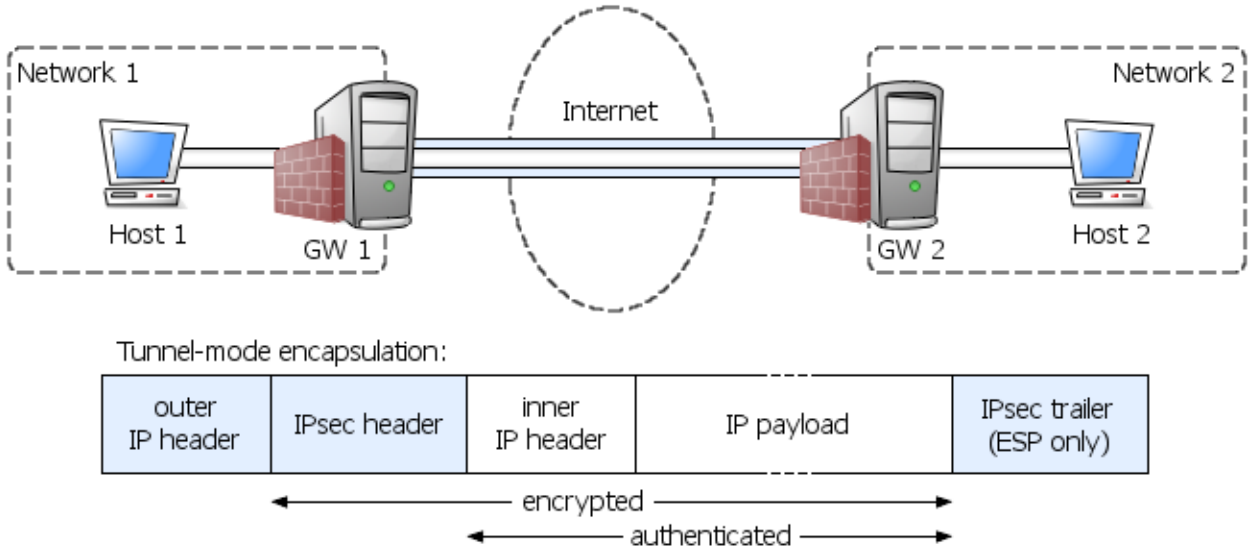
AH and ESP may also be applied in combination with each other to exploit the strengths of both protocols but, in most real-world scenarios, ESP alone is enough.

Both ESP and AH support two modes of operation:

transport mode
: IPsec protects only the payload of the IP packet (usually the transport layer data, hence its name), leaving the IP header, and thus routing, unchanged; transport mode can be used only for host-to-host communication;



tunnel mode
: the entire IP packet is encrypted and/or authenticated and then encapsulated into a new IP packet; tunnel mode is typically used to connect either two remote networks or a host and a network; it is more flexible than transport mode, but imposes more bandwidth overhead;



The flexibility of tunnel mode allows it to fully supersede the functionality of transport mode, at the reasonable expense of a slightly higher bandwidth overhead. As a consequence, transport mode is rarely used in real-world VPNs and, just like AH, [SCHNEIER] suggests that transport mode be eliminated altogether, with the advantage of significantly reducing IPsec complexity.

In a nutshell, while ESP and tunnel mode are by far the most prevalent choice, AH and transport mode can be considered the black sheeps of the IPsec protocol family!

2.2 SA, SPI, SPD and other acronyms

To actually establish the VPN, the IPsec protocols require that some state data be shared between the VPN endpoints, such as the cryptographic algorithms for encryption and authentication, the keys used as input to the cryptographic algorithms, the current sequence number, the antireplay window and so on.

These data are held in a data structure called a *Security Association* (SA); SAs are created by a specific protocol, IKEv2 (defined in [RFC4306]), which also has the responsibility of mutually authenticating the two communicating parties, setting up the encrypted channel for secure information exchange (these steps are part of the so-called *IKE phase 1*) and negotiating the shared secret from which cryptographic keys are derived (*IKE phase 2*).

A Security Association applies to a single protocol (AH or ESP) and to a single direction of traffic flow; therefore, "*to secure typical, bi-directional communication between two IPsec-enabled systems, a pair of SAs (one in each direction) is required. IKE explicitly creates SA pairs in recognition of this common usage requirement*" [RFC4301].

SAs are collected in a *Security Association Database* (SAD), where they are uniquely identified by the combination of protocol (AH or ESP), destination address and an arbitrary 32-bit value called the *Security Parameter Index* (SPI). The SPI has the specific task of helping the receiver to identify the SA under which an incoming packet should be processed.

But how does IPsec decide which datagrams to send through the VPN and which not? For instance, in a typical site-to-site VPN scenario, the IPsec gateway will usually tunnel and/or protect only traffic between the remote LANs, leaving all other traffic unaffected. Well, IPsec makes such decisions based on *policies*, i.e. user-defined rules stating which packets should be protected using IPsec security services, which should be allowed to bypass IPsec protection and which should be discarded. IPsec policies are applied based on some specific fields in the datagram headers, called *selectors*, which include: source and destination addresses, Next Layer Protocol, source and destination ports (if used by the next layer protocol).

As with Security Associations, IPsec policies are held in a database, called the *Security Policy Database* (SPD), which must be consulted during the processing of all traffic (inbound and outbound), including traffic not protected by IPsec, that traverses the IPsec boundary.

2.3 The life of an IPsec packet

To recap, let's have a look at what the (brief) life of an IPsec packet looks like; we will consider the most common case: an ESP tunnel-mode VPN between two remote networks (see picture above). The story begins when the first gateway (GW1) receives an outbound packet from a host (Host1) within its internal network and destined for a host (Host2) on the remote network:

- the gateway first compares the datagram's selector fields against the SPD to find the first matching policy;
- the policy may specify one of three possible processing choices:
  - DISCARD, the packet is not allowed to traverse the IPsec boundary and is dropped;
  - BYPASS, the packet is allowed to cross the IPsec boundary without IPsec protection and will be routed normally;
  - PROTECT, the packet must be afforded IPsec protection and the policy will point to zero or more SAs in the SAD;

  in the present case, the gateway has a policy specifying that the datagram must be encapsulated with tunnel-mode ESP and sent to GW2;
- if no SA exists for this policy, IKE will be invoked to negotiate the SAs with the appropriate peer;
- the first matching SA(s) will be applied, providing the requested security services to the datagram;
- the IP datagram will be encapsulated in ESP and the outer IP header will have the addresses of GW1 and GW2 as source and destination addresses respectively;

After a brief walk around the Internet, the encapsulated packet hits the second gateway (GW2):

- the datagram is checked to see whether it contains an IPsec header; if not, the datagram is forwarded normally;
- using the destination address, the SPI and the type of IPsec header of the incoming datagram, the gateway determines which SA to use; if no matching SA is found, the packet is dropped;

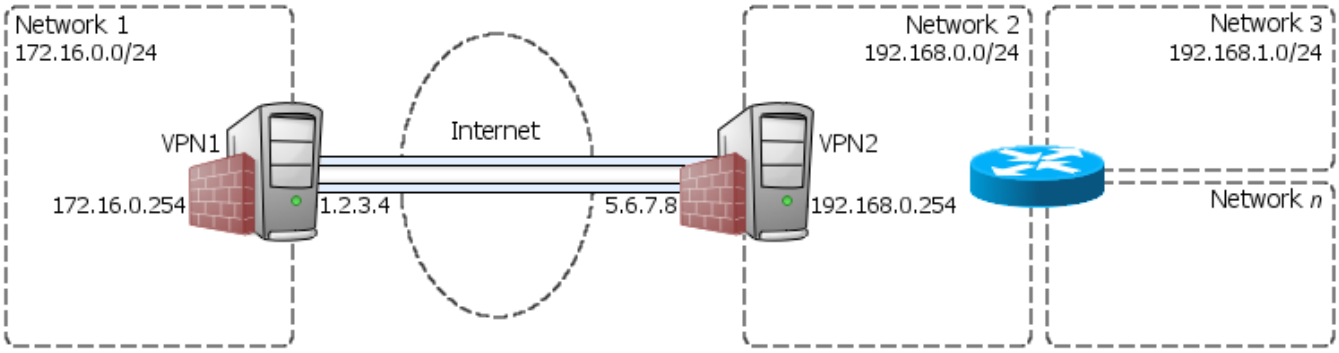
- if antireplay is activated, the sequence number is checked for validity;
- the packet is decrypted and/or authenticated as specified by the SA;
- the gateway locates the SPD entry that applies to the datagram based on its selectors and verifies that the SA(s) applied in the previous steps match with SA(s) specified by the policy;
- the packet is decapsulated and forwarded to next hop or to the appropriate transport protocol.

3. Ipsec on OpenBSD

Now that we have an adequate working knowledge of the IPsec architecture and protocols, we are finally ready to move from theory to practice and start having some fun with OpenBSD!

OpenBSD ships by default with full IPsec support in the stock kernel and provides a set of user-space daemons and tools for managing IPsec configuration, dynamic key exchange and high availability; and the great thing is that, as you'll see, setting up an IPsec VPN on OpenBSD is an incredibly simple and fast task, especially compared to most other IPsec implementations out there.

But before proceeding to edit configuration files and run system commands, let's take a brief look at the basic network topology of the VPN that we are going to set up in this document; it's a very simple site-to-site VPN, with a couple of multi-homed security gateways (VPN1 and VPN2) linking two remote private networks (172.16.0.0/24 and 192.168.0.0/24).



In this chapter, we will set up the VPN using IPsec: to be more precise, we will configure it in tunnel mode (the only option for network-to-network VPNs) and use the ESP protocol in order to encrypt the VPN traffic as it traverses the Internet; we will also consider the case of redundant IPsec gateways with carp(4). Then, in the next chapters, we will see how the same VPN can be implemented using alternative solutions, in particular OpenVPN and OpenSSH.

3.1 Preliminary steps

Before proceeding to configure IPsec, we have to perform a few preliminary steps to make sure the systems are correctly set up for IPsec to work properly. The IPsec protocols are enabled or disabled in the OS's TCP/IP stack via two `sysctl(3)` variables: `net.inet.esp.enable` and `net.inet.ah.enable`, both enabled by default; you can check this by running the `sysctl(8)` command:

```
# sysctl net.inet.esp.enable
net.inet.esp.enable=1
# sysctl net.inet.ah.enable
net.inet.ah.enable=1
```

Since our VPN gateways will have to perform traffic routing, we also need to enable IP forwarding, which is turned off by default. This is done, again, with `sysctl(8)`, by setting the value of the `net.inet.ip.forwarding` variable to "`1`" if you want any kind of traffic to be forwarded or "`2`" if you want to restrict forwarding to only IPsec-processed traffic:

```
# sysctl net.inet.ip.forwarding=1
net.inet.ip.forwarding: 0 -> 1
```

Optionally, you may also want to enable the IP Payload Compression Protocol (IPComp) to reduce the size of IP datagrams for higher VPN throughput; however, bear in mind that the reduction of bandwidth

usage comes at the expense of a higher computational overhead (see [RFC3173] for further details):

```
# sysctl net.inet.ipcomp.enable=1
net.inet.ipcomp.enable: 0 -> 1
```

To make these settings permanent across reboots, add the following variables to the /etc/sysctl.conf(5) file:

/etc/sysctl.conf

```
[ ... ]
net.inet.esp.enable=1      # Enable the ESP IPsec protocol
net.inet.ah.enable=1       # Enable the AH IPsec protocol
net.inet.ip.forwarding=1   # Enable IP forwarding for the host. Set it to '2' to
                           #   forward only IPsec traffic
net.inet.ipcomp.enable=1   # Optional: compress IP datagrams
```

Finally, we need to bring up the enc(4) virtual network interface. This interface allows you to inspect outgoing IPsec traffic before it is encapsulated and incoming IPsec traffic after it is decapsulated; this is primarily useful for filtering IPsec traffic with PF and for debugging purposes.

```
# ifconfig enc0 up
```

To make the system automatically bring up the enc(4) interface at boot, create the /etc/hostname.enc0 configuration file:

/etc/hostname.enc0

```
up
```

3.2 Setting up the PKI

OpenBSD's IKE key management daemon, isakmpd(8), relies on public key certificates for authentication and therefore requires that you first set up a Public Key Infrastructure (PKI) for managing digital certificates.

The first step in setting up the PKI is the creation of the root CA certificate (/etc/ssl/ca.crt) and private key (/etc/ssl/private/ca.key) on the signing machine (which doesn't have to be necessarily one of the VPN gateways) using openssl(1); e.g.:

```
CA# openssl req -x509 -days 365 -newkey rsa:1024 \
> -keyout /etc/ssl/private/ca.key \
> -out /etc/ssl/ca.crt
Generating a 1024 bit RSA private key
.....++++++
.....++++++
writing new private key to '/etc/ssl/private/ca.key'
Enter PEM pass phrase: <passphrase>
Verifying - Enter PEM pass phrase: <passphrase>
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) []: IT
State or Province Name (full name) []: Italy
Locality Name (eg, city) []: Milan
```

```
Organization Name (eg, company) []: Kernel Panic Inc.
Organizational Unit Name (eg, section) []: IPsec
Common Name (eg, fully qualified host name) []: CA.kernel-panic.it
Email Address []: danix@kernel-panic.it
CA#
```

The next step is the creation of a Certificate Signing Request (CSR) on each of the IKE peers; for instance, the following command will generate the CSR (`/etc/isakmpd/private/1.2.3.4.csr`) for the VPN1 machine (the IP address, in this case "1.2.3.4", is used as unique ID):

```
VPN1# openssl req -new -key /etc/isakmpd/private/local.key \
>  -out /etc/isakmpd/private/1.2.3.4.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) []: IT
State or Province Name (full name) []: Italy
Locality Name (eg, city) []: Milan
Organization Name (eg, company) []: Kernel Panic Inc.
Organizational Unit Name (eg, section) []: IPsec
Common Name (eg, fully qualified host name) []: 1.2.3.4
Email Address []: danix@kernel-panic.it

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []: <enter>
An optional company name []: <enter>
VPN1#
```

Next, the CSRs must be sent to the CA, which will generate the signed certificates out of the certificate requests. For instance, assuming the CSR file is in the current directory:

```
CA# env CERTIP=1.2.3.4 openssl x509 -req \
>  -days 365 -in 1.2.3.4.csr -out 1.2.3.4.crt \
>  -CA /etc/ssl/ca.crt -CAkey /etc/ssl/private/ca.key \
>  -CAcreateserial -extfile /etc/ssl/x509v3.cnf -extensions x509v3_IPAddr
Signature ok
subject=/C=IT/ST=Italy/L=Milan/O=Kernel Panic
Inc./OU=IPsec/CN=1.2.3.4/emailAddress=danix@kernel-panic.it
Getting CA Private Key
Enter pass phrase for /etc/ssl/private/ca.key: <passphrase>
CA#
```

Finally, you need to copy the newly-generated certificates (the files ending in `.crt`) to the respective machines in the `/etc/isakmpd/certs/` directory, as well as the CA certificate (`/etc/ssl/ca.crt`) in `/etc/isakmpd/ca/`.

3.3 Configuration

So we have conveniently set up the system for IPsec use and generated all the required certificates for IKE peer authentication; now we're finally ready to configure our VPN connection. On OpenBSD, all the configuration for IPsec takes place in a single file, `/etc/ipsec.conf(5)`, which uses a very compact syntax, similar to `pf.conf(5)`, to define almost every characteristic of the VPN; the basic format of the file is as follows:

- comment lines begin with a hash character ("#") and extend to the end of the line;
- rules may span across multiple lines using the backslash character ("\");
- network addresses can be specified in CIDR notation, as symbolic host names, interface names, or interface group names;
- to simplify the configuration file, macros can be used; macro names must start with a letter, may contain letters, numbers and underscores and must not be reserved words;
- certain parameters (such as IP addresses) can be expressed as lists; lists are comma-separated and enclosed in curly braces.

There are different types of `ipsec.conf(5)` rules, depending on whether you want IPsec flows and SAs to be set up automatically (using `isakmpd(8)`) or manually; we will only consider the former case (which is usually what you want), so please refer to the documentation for further details on manual setups. The syntax is as follows:

```
ike [mode] [encap] [tmode] [proto protocol] \
    from src [port sport] [(srcnat)] to dst [port dport] \
    [local localip] [peer remote] \
    [mode auth algorithm enc algorithm group group] \
    [quick auth algorithm enc algorithm group group] \
    [srcid string] [dstid string] \
    [psk string] [tag string]
```

Though it may look rather complex at first, actual rules are usually very short and simple because most of the parameters can be omitted, in which case the default values are used. But let's examine the rule syntax in detail:

`ike` [*mode*] [*encap*] [*tmode*]
: the `ike` keyword specifies that `isakmpd(8)` must be used to automatically establish the Security Associations for this flow; `mode` can be either "`active`" (`isakmpd(8)` will immediately start negotiation of this tunnel), "`passive`" (to wait for an incoming request from the remote peer to start negotiation) or "`dynamic`" (to be used for hosts with dynamic IP addresses) and defaults to "`active`"; `encap` specifies the encapsulation protocol and can be either "`esp`" (default) or "`ah`"; `tmode` is the transport mode to use, i.e. "`tunnel`" (default) or "`transport`".

`proto` *protocol*
: Restrict the flow to a specific IP protocol (e.g. TCP, UDP, ICMP); by default all protocols are allowed.

`from` *src* [`port` *sport*] [(*srcnat*)] `to` *dst* [`port` *dport*]
: Specify the source and destination addresses of the packets that this rule applies to; you may also specify source and/or destination ports, but only in conjunction with the TCP or UDP protocols. The `srcnat` parameter can be used to specify the actual source address in outgoing NAT/BINAT scenarios.

`local` *localip* `peer` *remote*
: Specify the local and remote endpoints of the VPN; the local endpoint is required only if the machine has multiple addresses; the remote endpoint can be omitted if it corresponds to the `dst` parameter.

mode `auth` *algorithm* `enc` *algorithm* `group` *group*
: Specify the mode ("`main`" or "`aggressive`") and cryptographic transforms to be used for IKE phase 1 negotiation; please refer to the documentation for a complete list of the possible values and their defaults.

`quick auth` *algorithm* `enc` *algorithm* `group` *group*
: Specify the cryptographic transforms to be used for IKE phase 2 negotiation; please refer to the documentation for a complete list of the possible values and their defaults.

`srcid` *string* `dstid` *string*
: Define the unique ID that `isakmpd(8)` will use as the identity of the local (`srcid`) and remote

(`dstid`) peer; if omitted, the IP address is used.

`psk` *string*
: Use a pre-shared key for authentication instead of `isakmpd(8)`.

`tag` *string*
: Add a pf(4) tag to IPsec packets matching this rule.

So let's write the configuration files for the site-to-site VPN we're setting up; as you'll see, it's a really trivial task and a few rules will do. On the VPN1 host, the `/etc/ipsec.conf(5)` file will look like this:

/etc/ipsec.conf

```
# Macros
ext_if      = "rl0"                                 # External interface (1.2.3.4)
local_net   = "172.16.0.0/24"                       # Local private network
remote_gw   = "5.6.7.8"                             # Remote IPsec gateway
remote_nets = "{192.168.0.0/24, 192.168.1.0/24}"    # Remote private networks

# Set up the VPN between the gateway machines
ike esp from $ext_if to $remote_gw
# Between local gateway and remote networks
ike esp from $ext_if to $remote_nets peer $remote_gw
# Between the networks
ike esp from $local_net to $remote_nets peer $remote_gw
```

and on VPN2:

/etc/ipsec.conf

```
# Macros
ext_if     = "rl0"                                  # External interface (5.6.7.8)
local_nets = "{192.168.0.0/24, 192.168.1.0/24}"     # Local private networks
remote_gw  = "1.2.3.4"                              # Remote IPsec gateway
remote_net = "172.16.0.0/24"                        # Remote private network

# Set up the VPN between the gateway machines
ike esp from $ext_if to $remote_gw
# Between local gateway and remote network
ike passive esp from $ext_if to $remote_net peer $remote_gw
# Between the networks
ike esp from $local_nets to $remote_net peer $remote_gw
```

Now we are ready to start the `isakmpd(8)` daemon on both gateways; we will make it run in the foreground ("`-d`" option) in order to easily notice any errors:

```
# isakmpd -K -d
```

Then, again on both gateways, we can parse `ipsec.conf(5)` rules ("`-n`" option of `ipsecctl(8)`) and, if no errors show up, load them:

```
# ipsecctl -n -f /etc/ipsec.conf
# ipsecctl -f /etc/ipsec.conf
```

You can check that IPsec flows and SAs have been correctly set up by running `ipsecctl(8)` with the "`-s all`" option; for example:

```
VPN1# ipsecctl -s all
FLOWS:
flow esp in from 192.168.0.0/24 to 1.2.3.4 peer 5.6.7.8 srcid 1.2.3.4/32 dstid
```

```
5.6.7.8/32 type use
flow esp out from 1.2.3.4 to 192.168.0.0/24 peer 5.6.7.8 srcid 1.2.3.4/32 dstid
5.6.7.8/32 type require
flow esp in from 192.168.1.0/24 to 1.2.3.4 peer 5.6.7.8 srcid 1.2.3.4/32 dstid
5.6.7.8/32 type use
flow esp out from 1.2.3.4 to 192.168.1.0/24 peer 5.6.7.8 srcid 1.2.3.4/32 dstid
5.6.7.8/32 type require
[ ... ]

SAD:
esp tunnel from 5.6.7.8 to 1.2.3.4 spi 0x027fa231 auth hmac-sha2-256 enc aes
esp tunnel from 1.2.3.4 to 5.6.7.8 spi 0x13ebc203 auth hmac-sha2-256 enc aes
esp tunnel from 1.2.3.4 to 5.6.7.8 spi 0x25da85ac auth hmac-sha2-256 enc aes
esp tunnel from 5.6.7.8 to 1.2.3.4 spi 0x891aa39b auth hmac-sha2-256 enc aes
[ ... ]
VPN1#
```

Well, since everything seems to be working fine, we can configure the system to automatically start the VPN at boot by adding the following variables in /etc/rc.conf.local(8) on both security gateways:

/etc/rc.conf.local

```
isakmpd_flags="-K"     # Avoid keynote(4) policy checking
ipsec=YES              # Load ipsec.conf(5) rules
```

3.4 Packet filtering

IPsec traffic can be filtered on the enc(4) interface, where it appears unencrypted before encapsualtion and after decapsulation. The following are the main points to keep in mind for filtering IPsec traffic:

- IPsec protocols (AH and/or ESP) must be explicitely allowed on the external interface; e.g.:

```
# Allow ESP encapsulated IPsec traffic on the external interface
pass in  on $ext_if proto esp from $remote_gw to $ext_if
pass out on $ext_if proto esp from $ext_if to $remote_gw
```

- isakmpd(8) requires that UDP traffic on ports 500 (isakmp) and 4500 (ipsec-nat-t) be allowed on the external interface; e.g.:

```
# Allow isakmpd(8) traffic on the external interface
pass in  on $ext_if proto udp from $remote_gw to $ext_if \
     port {isakmp, ipsec-nat-t}
pass out on $ext_if proto udp from $ext_if to $remote_gw \
     port {isakmp, ipsec-nat-t}
```

- if the VPN is in tunnel mode, IP-in-IP traffic between the two gateways must be allowed on the enc(4) interface: e.g.:

```
# Allow IP-in-IP traffic between the gateways on the enc(4) interface
pass in  on enc0 proto ipencap from $remote_gw to $ext_if keep state \
     (if-bound)
pass out on enc0 proto ipencap from $ext_if to $remote_gw keep state \
     (if-bound)
```

- as stated before, IPsec traffic filtering is done on the enc(4) interface, where it appears unencrypted. State on the enc(4) interface should be interface bound; e.g.:

```
# Filter unencrypted VPN traffic on the enc(4) interface
pass in  on enc0 from $remote_nets to $int_if:network keep state (if-bound)
```

```
pass out on enc0 from $int_if:network to $remote_nets keep state (if-bound)
```

3.5 Redundant VPNs with sasyncd(8)

One of the most interesting features of OpenBSD's implementation of the IPsec protocol is the possibility to set up multiple VPN gateways in a redundant configuration, allowing for transparent failover of VPN connections without any loss of connectivity.

Typically, in OpenBSD, redundancy at the network level is achieved through the carp(4) protocol, which allows multiple hosts on the same local network to share a common IP address. Redundancy at the logical VPN layer, instead, is provided by the sasyncd(8) daemon, which allows the synchronization of IPsec SA and SPD information between multiple IPsec gateways.

We have already covered the carp(4) protocol in a previous document about redundant firewalls, so we won't come back to this topic now; therefore, I assume that you already have a working carp(4) setup and that you have modified your configuration accordingly (in particular the ipsec.conf(5) and pf.conf(5) files).

Please note that, as stated in the documentation, "*for SAs with replay protection enabled, such as those created by isakmpd(8), the sasyncd(8) hosts must have pfsync(4) enabled to synchronize the in-kernel SA replay counters*" (for a detailed discussion of the pfsync(4) protocol, please refer to [CARP]).

The sasyncd(8) daemon is configured through the /etc/sasyncd.conf(5) file, which has a rather self-explanatory syntax; below is a sample configuration file:

/etc/sasyncd.conf

```
# carp(4) interface to track state changes on
interface carp0
# Interface group to use to suppress carp(4) preemption during boot
group     carp
# sasyncd(8) peer IP address or hostname. Multiple 'peer' statements are allowed
peer      172.16.0.253
# Shared AES key used to encrypt messages between sasyncd(8) hosts. It can be
# generated with the openssl(1) command 'openssl rand -hex 32'
sharedkey 0x115c413529ba5ac96b208d83a50473b3e6ade60e66c59a10a944ad3d273148dd
```

Since sasyncd.conf(5) contains the shared secret key used to encrypt data between the sasyncd(8) hosts, it should have restrictive permissions (600) and belong to the "root" or "_isakmpd" user:

```
# chown root /etc/sasyncd.conf
# chmod 600 /etc/sasyncd.conf
```

Well, now we're ready to run the sasyncd(8) daemon on the redundant gateways; but first we need to restart isakmpd(8) with the "-S" option, which is mandatory on redundant setups (remember to add it also to isakmpd_flags in /etc/rc.conf.local(8)):

```
# pkill isakmpd
# isakmpd -S -K
# sasyncd
```

You can use ipsecctl(8) to verify that SAs are correctly synchronized between the IPsec gateways. Finally, if everything is working fine, we only have to add the following variable to the /etc/rc.conf.local(8) file to automatically start sasyncd(8) on boot.

/etc/rc.conf.local

```
sasyncd_flags=""
```

Note: sasyncd(8) must be manually restarted every time isakmpd(8) is restarted.

4. OpenVPN

OpenVPN is "*a full-featured SSL VPN which implements OSI layer 2 or 3 secure network extension using the industry standard SSL/TLS protocol, supports flexible client authentication methods based on certificates, smart cards, and/or username/password credentials, and allows user or group-specific access control policies using firewall rules applied to the VPN virtual interface*" [OVPN-HOWTO]. Its cross-platform portability, renown security and ease of use have made OpenVPN one of the most popular VPN solutions today.

Unlike IPsec, OpenVPN is not tightly integrated into the Operating System's kernel, but runs as a user-mode daemon and communicates with the TCP/IP stack via a `tun(4)` pseudo-device. Please refer to [OVPN-SEC2] for a detailed overview of the OpenVPN protocol and security model.

In the next paragraphs, we will implement the same VPN topology as in the previous chapter, though replacing IPsec with OpenVPN. The VPN1 machine will act as the server and wait for incoming connections from VPN2.

4.1 Installation and configuration

OpenVPN installation simply requires adding a couple of packages on both server and client(s):

- lzo-*xx*.tgz
- openvpn-*x.x*.tgz

4.1.1 Setting up the PKI

The first step in configuring OpenVPN is to set up the Public Key Infrastructure, by creating:

- a root CA certificate and private key;
- a certificate and private key for the OpenVPN server;
- a separate certificate and private key for each client that will connect to the VPN.

The CA private key will be used to sign the server and client certificates; this will allow the two VPN endpoints to mutually authenticate each other simply by verifying the CA signature of the other party's certificate, without having to previously know any other certificate but their own (see [OVPN-PKI] for further details).

OpenVPN provides a set of scripts, located in `/usr/local/share/examples/openvpn/easy-rsa/2.0/`, that greatly simplify the process of creating and managing the PKI. These scripts require, as a preliminary step, that you initalize a bunch of parameters in the `vars` file with your organization's data, to avoid being prompted for the same information every time you create a new certificate:

```
/usr/local/share/examples/openvpn/easy-rsa/2.0/vars
```

```
export EASY_RSA="`pwd`"

export OPENSSL="openssl"
export PKCS11TOOL="pkcs11-tool"
export GREP="grep"

export KEY_CONFIG="$EASY_RSA/openssl.cnf"
export KEY_DIR="$EASY_RSA/keys"

echo NOTE: If you run ./clean-all, I will be doing a rm -rf on $KEY_DIR

export PKCS11_MODULE_PATH="dummy"
export PKCS11_PIN="dummy"

export KEY_SIZE=1024
```

```
export CA_EXPIRE=3650
export KEY_EXPIRE=3650

export KEY_COUNTRY="IT"
export KEY_PROVINCE="Italy"
export KEY_CITY="Milan"
export KEY_ORG="Kernel Panic Inc."
export KEY_EMAIL="danix@kernel-panic.it"
```

Now, after sourcing the `vars` file, you can initialize the PKI by building the Diffie-Hellman parameters and creating the root CA certificate and key:

```
# cd /usr/local/share/examples/openvpn/easy-rsa/2.0/
# . ./vars
NOTE: when you run ./clean-all, I will be doing a rm -rf on
/usr/local/share/example/opevvpn/easy-rsa/2.0/keys
# ./clean-all
# ./build-dh
Generating DH parameters, 1024 bit long safe prime, generator 2
This is going to take a long time
[ ... ]
# ./pkitool --initca
Using CA Common Name: Kernel Panic Inc. CA
Generating a 1024 bit RSA private key
.........................++++++
......++++++
writing new private key to 'ca.key'
-----
#
```

The next step is creating the certificate and key for the VPN server:

```
# ./pkitool --server vpn1.kernel-panic.it
Generating a 1024 bit RSA private key
........++++++
.........................................++++++
writing new private key to 'vpn1.kernel-panic.it.key'
-----
Using configuration from /usr/local/share/examples/openvpn/easy-rsa/2.0/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'IT'
stateOrProvinceName   :PRINTABLE:'Italy'
localityName          :PRINTABLE:'Milan'
organizationName      :PRINTABLE:'Kernel Panic Inc.'
commonName            :PRINTABLE:'vpn1.kernel-panic.it'
emailAddress          :IA5STRING:'danix@kernel-panic.it'
Certificate is to be certified until Jun  2 08:41:51 2019 GMT (3650 days)

Write out database with 1 new entries
Data Base Updated
#
```

Next, we will use the `pkitool` utility to generate as many client certificates as we need:

```
# ./pkitool vpn2.kernel-panic.it
Generating a 1024 bit RSA private key
...................++++++
..++++++
writing new private key to 'vpn2.kernel-panic.it.key'
-----
```

```
Using configuration from /usr/local/share/examples/openvpn/easy-rsa/2.0/openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'IT'
stateOrProvinceName   :PRINTABLE:'Italy'
localityName          :PRINTABLE:'Milan'
organizationName      :PRINTABLE:'Kernel Panic Inc.'
commonName            :PRINTABLE:'vpn2.kernel-panic.it'
emailAddress          :IA5STRING:'danix@kernel-panic.it'
Certificate is to be certified until Jun  2 08:47:25 2019 GMT (3650 days)

Write out database with 1 new entries
Data Base Updated
#
```

So we have generated all the certificates and keys we need; you can find them in the `/usr/local/share/examples/openvpn/easy-rsa/2.0/keys` directory, ready to be copied to the appropriate machines. But before proceeding to copy the key files, we need to create, on both server and clients, the directory (`/etc/openvpn/private`) that will contain the private keys and assign it restrictive permissions to prevent unauthorized access.

```
# mkdir -p /etc/openvpn/private
# chmod 700 /etc/openvpn/private
```

The following are the files that must be copied from the CA-signing machine to the OpenVPN hosts:

- the `ca.crt` file (the CA certificate) must be copied to the `/etc/openvpn` directory of all the machines (server and clients);
- the `ca.key` file (the CA private key) must reside only on the key-signing machine; if you want the OpenVPN server to act also as the CA, just move this file to the `/etc/openvpn/private/` directory of the server machine;
- the `dh1024.pem` file (the Diffie Hellman parameters) must be placed in the `/etc/openvpn` directory of the server machine;
- the remaining `.crt` and `.key` files (i.e. the certificates and private keys of the server and the clients) must be copied to the respective machines; private keys must be stored in `/etc/openvpn/private` and certificates should reside in `/etc/openvpn`.

Finally, remember to delete all the files in `/usr/local/share/examples/openvpn/easy-rsa/2.0/keys/`:

```
# ./clean-all
```

4.1.2 Server configuration

OpenVPN supports a number of configuration parameters, allowing you to deeply customize its behaviour. These parameters can be either passed from the command-line or in a configuration file. Omitted parameters take the default value.

Below is a sample configuration file (see [OVPN-MAN] for a complete list of all the available parameters):

`/etc/openvpn/server.conf`

```
# Transport protocol to use. Available protocols are udp and tcp-server
proto udp
# TCP/UDP port to bind to
port 1194
```

```
# Name of the tun(4) device to use
dev tun0

# Uncomment to enable the management interface on port 1195. The password file
# only contains the management password on a single line.
#management 127.0.0.1 1195 /etc/openvpn/private/mgmt.pwd

# Path to the CA certificate
ca /etc/openvpn/ca.crt
# Path to the server's certificate file
cert /etc/openvpn/vpn1.kernel-panic.it.crt
# Path to the private key file
key /etc/openvpn/private/vpn1.kernel-panic.it.key
# Path to the file containing the Diffe-Hellman parameters
dh /etc/openvpn/dh1024.pem

# Address range for the tun(4) interfaces
server 10.0.1.0 255.255.255.0
# Uncomment to allow clients to dynamically change address (useful for
# road-warriors)
#float

# Send periodic keepalive messages
keepalive 10 120
# Use lzo compression to reduce network utilization
comp-lzo

# User the OpenVPN daemon should run as
user _openvpn
# Group the OpenVPN daemon should run as
group _openvpn
# Make the server daemonize after initialization
daemon openvpn

# Don't re-read key files upon receiving a SIGUSR1 signal
persist-key
# Don't close and reopen the tun(4) device upon receiving a SIGUSR1 signal
persist-tun

# Add a route to the local network to the client's routing table
push "route 172.16.0.0 255.255.255.0"
# Add routes to the remote networks to the server's routing table
route 192.168.0.0 255.255.255.0
route 192.168.1.0 255.255.255.0
# Directory for client-specific configuration files
client-config-dir /etc/openvpn/ccd

# Uncomment to periodically write status information to the specified file
#status /var/log/openvpn-status.log
# Uncomment to raise verbosity level for debugging
#verb 11
```

The `client-config-dir` directive in the server configuration file allows you to specify a directory containing client-specific configuration files. These files must have have the same name as the client's X509 Common Name, specified during the creation of the certificates. In this case, we will create a file named `/etc/openvpn/ccd/vpn2.kernel-panic.it`, which will specify which private networks can be reached through the OpenVPN client:

```
/etc/openvpn/ccd/vpn2.kernel-panic.it
```

```
iroute 192.168.0.0 255.255.255.0
iroute 192.168.1.0 255.255.255.0
```

Though very similar, both the `route` and `iroute` directives are necessary, because "*`route` controls the routing from the kernel to the OpenVPN server (via the `tun(4)` interface) while `iroute` controls the routing from the OpenVPN server to the remote clients*" [OVPN-HOWTO].

4.1.3 Client configuration

The client-side configuration is pretty similar to server-side configuration. The address and port of the server are specified via the `remote` directive. Make sure that the configuration matches the server configuration, in particular that they both use the same protocol, device type and that they both enable or disable lzo compression.

`/etc/openvpn/client.conf`

```
# Act as a client
client
# IP address (or hostname) and port of the OpenVPN server. You may specify
# multiple 'remote' options for redundancy.
remote 1.2.3.4 1194

# Transport protocol to use. Available protocols are udp and tcp-client
proto udp
# Name of the tun(4) device to use
dev tun0

# Uncomment if you connect through an HTTP proxy. The authfile must contain
# user and password on 2 lines. The authentication type can be 'none', 'basic'
# or 'ntlm'
#http-proxy proxy_addr proxy_port /etc/openvpn/private/authfile auth_type
# Make the server daemonize after initialization
daemon openvpn
# Send periodic keepalive messages
keepalive 10 120

# Don't bind to the local address and port, i.e. don't wait for incoming
# connections
nobind

# User the OpenVPN daemon should run as
user _openvpn
# Group the OpenVPN daemon should run as
group _openvpn
# Directory to chroot to after initialization
chroot /var/empty

# Don't re-read key files upon receiving a SIGUSR1 signal
persist-key
# Don't close and reopen the tun(4) device upon receiving a SIGUSR1 signal
persist-tun

# Path to the CA certificate
ca /etc/openvpn/ca.crt
# Path to the client's certificate file
cert /etc/openvpn/vpn2.kernel-panic.it.crt
# Path to the private key file
key /etc/openvpn/private/vpn2.kernel-panic.it.key

# Require that the peer certificate has the nsCertType field set to 'server'
ns-cert-type server
# Use lzo compression to reduce network utilization
comp-lzo

# Uncomment to periodically write status information to the specified file
```

```
#status /var/log/openvpn-status.log
# Uncomment to raise verbosity level for debugging
#verb 11
```

4.2 Starting the VPN

Before starting the VPN, we have to enable IP forwarding on both gateways, since they will have to perform routing of network traffic:

```
# sysctl net.inet.ip.forwarding=1
```

Uncomment the following line in /etc/sysctl.conf(5) to re-enable IP forwarding after reboot:

/etc/sysctl.conf

```
net.inet.ip.forwarding=1
```

So we're ready to start the VPN! Just run the following command on the server:

```
vpn1# openvpn --config /etc/openvpn/server.conf
```

and the following on the client:

```
vpn2# openvpn --config /etc/openvpn/client.conf
```

To finish, we just have to create the configuration file for the tun(4) interface on the server (starting OpenVPN from this file improves compatibility with PF):

/etc/hostname.tun0

```
up
!/usr/local/sbin/openvpn --daemon --config /etc/openvpn/server.conf
```

and on the client:

/etc/hostname.tun0

```
up
!/usr/local/sbin/openvpn --daemon --config /etc/openvpn/client.conf
```

5. OpenSSH

OpenSSH is "*a FREE version of the SSH connectivity tools*" developed by the OpenBSD project. It certainly needs no introduction as it has now grown into the de facto standard for secure console access over the Internet, widely supplanting the infamous "r" commands.

Beginning with version 4.3, OpenSSH also provides secure VPN tunneling capabilities at both layer 2 and layer 3 of the OSI model, by using the `tun(4)` pseudo-device to encapsulate network traffic within SSH packets.

Of the VPN solutions we've seen so far, OpenSSH-based VPNs are by far the simplest to use and the fastest to implement; however, they also imply a considerable overhead. As a consequence, the documentation warns that OpenSSH VPNs "*may be more suited to temporary setups, such as for wireless VPNs*", and recommends the use of IPsec for more permanent VPNs.

5.1 Configuration

We will configure the same VPN topology as in the previous chapters; the VPN1 machine will act as the OpenSSH server, waiting for connections from VPN2.

First off, we need to enable tunneling support on the OpenSSH server, since this feature is disabled by default. This is achieved by setting the `PermitTunnel` parameter in `/etc/ssh/sshd_config(5)` to `ethernet` or `point-to-point`, depending on whether you want the VPN to operate, respectively, at layer 2 of the OSI model, layer 3 or both.

```
/etc/ssh/sshd_config
```

```
[ ... ]

# Enable layer-3 tunneling. Change the value to 'ethernet' for layer-2 tunneling
PermitTunnel point-to-point
```

On the client side, the `Tunnel` parameter, in `/etc/ssh/ssh_config(5)`, must be set to the same value as `PermitTunnel` on the OpenSSH server:

```
/etc/ssh/ssh_config
```

```
[ ... ]

# Enable layer-3 tunneling. Change the value to 'ethernet' for layer-2 tunneling
Tunnel point-to-point
```

Next, we need to enable IP forwarding on both VPN gateways, since they will have to perform routing of network traffic:

```
# sysctl net.inet.ip.forwarding=1
```

Uncomment the following line in `/etc/sysctl.conf(5)` to re-enable it after reboot:

```
/etc/sysctl.conf
```

```
net.inet.ip.forwarding=1
```

And the configuration phase is over: how could it be easier? Now we only have to force `sshd(8)` to reread its configuration file by sending it a SIGHUP signal:

```
VPN1# pkill -HUP sshd
```

5.2 Starting the VPN

Before actually firing up the VPN, we will carry out a couple of preliminary steps on both the OpenSSH server and the client, i.e. creating and configuring the `tun(4)` network device and setting up the appropriate routes to the remote network(s) and hosts.

```
VPN1# ifconfig tun0 create
VPN1# ifconfig tun0 10.0.0.1 10.0.0.2 netmask 0xfffffffc
VPN1# route add 192.168.0.0/24 10.0.0.2
VPN1# route add 192.168.1.0/24 10.0.0.2
```

```
VPN2# ifconfig tun0 create
VPN2# ifconfig tun0 10.0.0.2 10.0.0.1 netmask 0xfffffffc
VPN2# route add 172.16.0.0/24 10.0.0.1
```

Well, we're finally ready to initiate the `ssh(1)` connection and establish the VPN tunnel. The `-f` option requests `ssh(1)` to go to background after prompting for the password, and the `-w` option specifies the numerical ID of the `tun(4)` device in charge of forwarding VPN traffic; in our setup, we're using `tun0` on both client and server, so we will set this option to `0:0`.

```
VPN2# ssh -f -w 0:0 1.2.3.4 true
root@VPN1's password: pAssWOrd
```

5.3 Finishing touches

To finish, we will configure the client machine to automatically start the VPN on boot. To prevent the system from hanging during startup until the user enters the password, we need to create an RSA authentication key for the user with the `ssh-keygen(1)` utility:

```
VPN2# ssh-keygen -b 2048 -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa): <enter>
Enter passphrase (empty for no passphrase): <enter>
Enter same passphrase again: <enter>
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
cd:9c:3b:3a:c0:92:7f:c2:9b:6e:3a:48:dc:50:a4:2a root@vpn2.kernel-panic.it
VPN2#
```

and add the newly-generated key, contained in `/root/.ssh/id_rsa.pub`, to the authorized keys in `/root/.ssh/authorized_keys` on the server; please make sure that this file has restrictive permissions (`600`):

```
VPN1# cat authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAoWqL6wpgL5j3dlFtdYWT+cc72F/FtMhmTBLUEcCMQQy8
/V9CptSn7yCC+1R5xhZD8WO3d11c7R8pUHPP77A3omFruEpk4pREuisHnMtA6XyVFoxshhV1osyoQ/HJ
w6BhTmmGDCCyNsPmQyAPi9V7rL4NNSlll6mFXqLDNth6wf0qjo33BURsyKR6xxmt5QBhDpCBDel3EwLh
gE2Jy06XJZKa62/WU6ofbnXZWwGX8ZsbCPxqqu3EOBhMwlUgA1IgksGfOcB4rgV+qpcPUfl3fQM67Mc7
Nwhh7jqkaCTpu/vs4OpBFt6j9eVxMgRGylg4a9tBcZY2588wPZZThpx/sw== root@vpn2.kernel-pa
nic.it
VPN1# chmod 600 /root/.ssh/authorized_keys
```

Next, on the server side, we need to create the configuration file for the `tun(4)` pseudo-device, `/etc/hostname.tun0`, which will also include the necessary static routes:

/etc/hostname.tun0

```
10.0.0.1 10.0.0.2 netmask 0xfffffffc
!route add 192.168.0.0/24 10.0.0.2 >/dev/null 2>&1
!route add 192.168.1.0/24 10.0.0.2 >/dev/null 2>&1
```

Similarly, on the client side, we will create the `/etc/hostname.tun0` configuration file :

/etc/hostname.tun0

```
10.0.0.2 10.0.0.1 netmask 0xfffffffc
!route add 172.16.0.0/24 10.0.0.1 >/dev/null 2>&1
```

but also add the VPN start command in `/etc/rc.local(8)`.

/etc/rc.local

```
[ ... ]
echo -n ' OpenSSH-VPN'
/usr/bin/ssh -f -w 0:0 1.2.3.4 true
```

6. Appendix

6.1 References

- [OBSD39] - The OpenBSD 3.9 Release
- [OVPN-FAQ] - OpenVPN FAQ, What is the difference between bridging and routing?
- [OVPN-SEC] - Are there any known security vulnerabilities with OpenVPN?
- [RFC4301] - RFC 4301, Security Architecture for the Internet Protocol
- [RFC4302] - RFC 4302, IP Authentication Header
- [RFC4303] - RFC 4303, IP Encapsulating Security Payload (ESP)
- [SCHNEIER] - *A Cryptographic Evaluation of IPsec*, N. Ferguson and B. Schneier
- [RFC4306] - RFC 4306, Internet Key Exchange (IKEv2) Protocol
- [RFC3173] - RFC 3173, IP Payload Compression Protocol (IPComp)
- [CARP] - Redundant firewalls with OpenBSD, CARP and pfsync
- [OVPN-HOWTO] - OpenVPN 2.0 HOWTO
- [OVPN-SEC2] - OpenVPN Security Overview
- [OVPN-PKI] - Setting up your own Certificate Authority (CA) and generating certificates and keys for an OpenVPN server and multiple clients
- [OVPN-MAN] - OpenVPN 2.0.x Man Page

6.2 Bibliography

- *VPNs Illustrated: Tunnels, VPNs, and IPsec*, Jon C. Snader, Addison Wesley, 2006